

Kathmandu University

School of Engineering

Department of Artificial Intelligence

Dhulikhel, Kavre

Semester Project Final Report

In partial fulfillment of the requirements for
IV Year, I Semester of B.Tech in Artificial Intelligence

MarketGyan: NEPSE and Financial News Analysis

An Extension of Khabar AI

Submitted by: Bikash Kumar Sah

Roll Number: 16

Submission Date: July 3, 2026

Submitted to

Head of Department

Department of Artificial Intelligence

Abstract

MarketGyan is a Nepal-focused financial natural language processing project that extends the Khabar AI news platform into a market-analysis assistant for the Nepal Stock Exchange (NEPSE). The project combines public market snapshots, local financial news, regulatory documents, a bilingual human-adjudicated benchmark, supervised baselines, a fine-tuned Qwen3.5-9B LoRA adapter, and a retrieval-augmented generation runtime. Its main research artifact is NEPSE-Impact-500: a 500-row English and Nepali benchmark for identifying whether Nepal-focused news is relevant to NEPSE, classifying event type and potential market direction, mapping entities to sectors and symbols, and grounding each label in numbered evidence sentences.

The project first trained a multilingual XLM-R baseline on the frozen benchmark. XLM-R relevance classification reached 0.787 accuracy and 0.723 macro-F1 on the 75-row test split, while relevant-record direction classification reached 0.683 accuracy and 0.549 macro-F1. A small English-only FinBERT sanity baseline reached 0.889 accuracy and 0.616 macro-F1 on the English relevant subset. The final Qwen3.5-9B targeted-v2 adapter improved the structured extraction path, reaching 1.000 strict structured-output validity, 1.000 evidence grounding, 0.840 relevance accuracy, and 0.796 relevance macro-F1 on the same frozen test split. The live single-pass RAG agent passes citation, disclaimer, and no-advice scenario checks, and the frontend demonstrates grounded Qwen answers, evidence search, and local report generation. At the same time, retrieval Precision@5, neutral direction recall, sector recall, symbol recall, and evidence-recall remain weak. The final contribution is therefore best understood as a demo-ready research prototype with a reproducible benchmark and clear technical limitations, not as a production financial advisory system.

Acknowledgement

I would like to express sincere gratitude to the Department of Artificial Intelligence, Kathmandu University, for providing the academic structure for this semester project. I am especially thankful to my supervisors, Mr. Sundeep Gupta and Ms. Sristi Khatiwada, for their guidance, review, and support throughout the development of MarketGyan. Their feedback helped narrow the project from a broad financial assistant into a measurable Nepal-specific financial NLP and retrieval system.

I am also thankful to the faculty members and project reviewers whose comments improved the benchmark, evaluation plan, and final system demonstration. Public market and regulatory sources, including NEPSE, ShareSansar, Nepal Rastra Bank, and SEBON, made the research prototype possible through publicly available data. Finally, I would like to thank everyone who supported the annotation, testing, and review process during the final project phase.

Supervisor Signature

Supervisor Signature

Mr. Sundeep Gupta

Ms. Sristi Khatiwada

Date: _____

Date: _____

Contents

Abstract	i
Acknowledgement	ii
List of Figures	vi
List of Tables	vii
List of Abbreviations	viii
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope	2
2 Related Work	4
2.1 Nepali and Multilingual NLP	4
2.2 Financial NLP and Financial LLMs	4
2.2.1 FinBERT: Domain-Specific Financial Sentiment	5
2.2.2 FinGPT: Data-Centric Financial LLM Adaptation	5
2.2.3 FinLLM Survey: Benchmarks, Hallucination, and Evaluation	5
2.2.4 FinTral: Multi-Task Financial Instruction Tuning	5

2.3	Parameter-Efficient Fine-Tuning	6
2.4	Retrieval-Augmented Generation and Citations	6
3	Methodology	7
3.1	System Overview	7
3.2	Data Acquisition Method	8
3.3	Data Pipeline and Storage Contract	8
3.4	Benchmark Construction	9
3.5	Schema-v2 Curation and Structured Output Contract	9
3.6	Training Flow	11
3.7	Modeling	11
3.8	Runtime Design	12
4	System Requirement Specification	13
4.1	Functional Requirements	13
4.2	Non-Functional Requirements	13
4.3	Resource Requirements	14
4.4	XLm-R Training Configuration	14
4.5	Qwen3.5-9B Training Configuration	14
5	Implementation	16
5.1	Backend Data Services	16
5.2	Annotation and Validation Tools	16
5.3	Vector Indexing and Retrieval	16
5.4	Model Serving	17
5.5	Single-Pass Agent and Reports	17
5.6	Frontend Demonstration	18

5.7	Safety and Fail-Closed Behavior	18
6	Results and Evaluation	19
6.1	Baseline Model Results	19
6.2	Qwen Model-Only Benchmark	20
6.3	Qwen Prompting and Fine-Tuning Ablation	21
6.4	Constrained Runtime Inference	22
6.5	Retrieval Evaluation	23
6.6	Scenario and Report Evaluation	23
7	Discussion	25
7.1	Data and Benchmark Findings	25
7.2	Modeling Findings	25
7.3	Retrieval and Runtime Findings	26
7.4	Final Project Status	26
8	Limitations and Future Work	27
8.1	Limitations	27
8.2	Future Work	27
9	Conclusion	29
A	Demo Screenshots	30

List of Figures

3.1	Training and evaluation workflow.	7
3.2	Daily inference workflow.	7
6.1	XLMM-R relevance baseline result plot exported from the v3 baseline artifact.	20
6.2	XLMM-R direction baseline result plot exported from the v3 baseline artifact.	20
6.3	Qwen3.5-9B targeted-v2 training and evaluation loss curve from the latest A100 output artifact.	21
6.4	Qwen3.5-9B targeted-v2 frozen-test summary plot exported with the final adapter artifact.	21
A.1	MarketGyan overview tab showing market snapshot, live runtime checks, and Qdrant collection status.	30
A.2	Ask MarketGyan tab with a proposal-style market query form. . . .	31
A.3	Generated Qwen answer with disclaimer, sentence-grounded citations, and source evidence.	31
A.4	Published MarketGyan report with headline, market summary, sector analysis, and report citations.	32

List of Tables

3.1	Modeling roles used in MarketGyan.	11
3.2	Runtime component responsibilities.	12
4.1	Final resource requirements and actual runtime choices.	14
4.2	Implemented XLM-R baseline training configuration.	14
4.3	Final Qwen3.5-9B targeted-v2 BF16 LoRA training configuration. .	15
6.1	XLM-R and FinBERT baseline results.	19
6.2	Strict frozen-test comparison for Qwen3.5-9B A100 runs.	20
6.3	Qwen zero-shot, few-shot, and fine-tuned ablation on the frozen test split.	22
6.4	Qwen evidence-selection ablation.	22
6.5	Model-only adapter output versus vLLM JSON Schema constrained inference.	22
6.6	Proposal retrieval evaluation after manual labels.	23
6.7	Final proposal-aligned system readout.	24

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
BF16	Brain Floating Point 16-bit
F1	F1-Score
JSON	JavaScript Object Notation
LLM	Large Language Model
LoRA	Low-Rank Adaptation
NEPSE	Nepal Stock Exchange
NLP	Natural Language Processing
NRB	Nepal Rastra Bank
P@5	Precision at 5
QLoRA	Quantized Low-Rank Adaptation
RAG	Retrieval-Augmented Generation
SEBON	Securities Board of Nepal
UI	User Interface
vLLM	Virtual Large Language Model serving engine

Chapter 1

Introduction

1.1 Background

Investors in Nepal follow daily market movement through NEPSE market summaries, broker portals, financial news websites, regulatory circulars, and social media discussions. These sources are fragmented. A user who wants to understand a single trading day must manually connect index movement, sector performance, company-specific announcements, monetary policy, regulation, and news sentiment. General news summarizers are not enough for this task because financial interpretation depends on market relevance, listed-company identification, event type, potential direction, and evidence quality.

MarketGyan extends the existing Khabar AI platform into this financial-analysis domain. Instead of building a separate application from the beginning, the project reuses the existing news ingestion, scheduler, backend, semantic search, and React frontend foundations. The new contribution is a finance-specific data pipeline, benchmark, model, retrieval contract, citation validation path, and local demonstration workflow.

1.2 Problem Statement

The project problem is to design and evaluate a Nepal-specific financial NLP system that can combine structured market data with local financial news and regulatory updates. The system should identify which articles are relevant to NEPSE, classify their likely market impact, retrieve supporting evidence, and generate grounded answers or daily reports without giving buy or sell advice. The work is a research

prototype and demonstration system, not a production trading platform.

1.3 Objectives

The project objectives were:

1. extend Khabar AI to collect and manage NEPSE-related market data, financial news, and selected regulatory sources;
2. create a bilingual, human-adjudicated benchmark for NEPSE news relevance and potential impact;
3. fine-tune and evaluate a 7B–10B class instruction model for structured financial impact extraction;
4. build a RAG runtime that answers user queries and generates reports with evidence citations; and
5. evaluate model quality, retrieval quality, grounding, report behavior, latency, and safety against the proposal requirements.

1.4 Scope

The system uses public market data, public news, and public regulatory sources. It covers daily NEPSE market snapshots, sector-level market context, finance-focused news articles, selected NRB and SEBON regulatory material when machine-readable text is available, and archived MarketGyan reports. The model scope is limited to news relevance, event type, potential impact direction, affected sectors or symbols, and evidence sentence selection. The application scope includes evidence search, grounded question answering, and local report generation inside the existing Khabar AI dashboard.

The project deliberately excludes trading execution, brokerage integration, portfolio optimization, personal risk profiling, price forecasting, and direct buy or sell recommendations. It also excludes unreadable legacy-font PDFs and OCR-heavy regulatory extraction from the final deliverable because those files require a separate document-processing track. MarketGyan is therefore an informational research

prototype: it can explain what public evidence says about market events, but it does not claim to predict returns or provide investment advice.

The evaluation scope is also bounded. The frozen 500-row benchmark measures model behavior on curated English and Nepali article examples. The retrieval benchmark measures whether the current Qdrant index can return relevant evidence for 30 proposal-style queries. The scenario benchmark measures citation correctness, disclaimer behavior, and fail-closed operation for report/query workflows. These tests are sufficient for an academic prototype evaluation, but they are not equivalent to production reliability testing across all Nepal-market news sources and trading dates.

Chapter 2

Related Work

2.1 Nepali and Multilingual NLP

Work on Nepali NLP remains more limited than work on high-resource languages, but recent studies provide a stronger foundation than was available even a few years ago. Timilsina et al. introduced NepBERTa, a monolingual Nepali language model trained on a large news corpus, and showed that language-specific pre-training improves multiple Nepali NLP tasks [5]. Subedi et al. evaluated large language models for Nepali named-entity recognition and part-of-speech tagging, highlighting both the promise of modern transformer models and the continuing importance of low-resource benchmarks [6]. Thapa et al. expanded the Nepali model landscape by training new encoder and decoder models on a substantially larger monolingual corpus and reporting gains on both understanding and generation tasks [7]. These studies suggest that Nepali text can now be processed more directly in applied systems, especially when multilingual or Nepali-adapted models are combined with careful fine-tuning and task-specific evaluation. MarketGyan follows this direction by treating Nepali and English financial articles as first-class benchmark inputs rather than translating everything into English.

2.2 Financial NLP and Financial LLMs

Financial text differs from general news because it contains domain-specific terminology, event-driven sentiment, and interpretation that depends on policy, regulation, and sector context. MarketGyan is positioned within this financial NLP line of work, but it narrows the problem to Nepal-market relevance, potential impact, and evidence-grounded structured extraction.

2.2.1 FinBERT: Domain-Specific Financial Sentiment

FinBERT established that adapting a transformer model to financial language improves financial sentiment classification compared with using a general-purpose language model directly [1]. This is important for MarketGyan because the same word can imply different direction depending on financial context. For example, a capital raise, a regulatory notice, or a liquidity update cannot be judged only by generic positive or negative wording. The project therefore includes an English-only FinBERT sanity baseline while recognizing that it cannot cover Nepali articles or the full structured NEPSE impact schema.

2.2.2 FinGPT: Data-Centric Financial LLM Adaptation

FinGPT emphasizes that financial LLM performance depends heavily on data curation, lightweight adaptation, and task-specific instruction data [2]. MarketGyan adopts this data-centric view. The main contribution is not the LoRA method itself, but the curated NEPSE-Impact-500 benchmark, hard-negative relevance examples, sentence-grounded evidence labels, and Nepal-specific event ontology. This choice also explains why the final improvement path focuses on benchmark quality, retrieval metadata, and evidence selection rather than simply increasing model size.

2.2.3 FinLLM Survey: Benchmarks, Hallucination, and Evaluation

The FinLLM survey summarizes the rapid movement from financial sentiment classifiers to broader financial language models, while warning that hallucination, weak benchmarks, and incomplete evaluation remain central risks [3]. This directly influenced the MarketGyan evaluation design. The final report separates structured validity, grounding, relevance, event type, direction, sector/symbol selection, evidence selection, retrieval Precision@5, citation correctness, latency, and safety checks instead of reporting only a single accuracy number.

2.2.4 FinTral: Multi-Task Financial Instruction Tuning

FinTral demonstrates financial instruction tuning across multiple financial tasks and modalities using a moderately sized open model family [4]. MarketGyan uses a narrower but similar principle: the Qwen3.5-9B adapter is trained to produce one

compact multi-field output that combines relevance, event type, scope, direction, mechanism, sectors, symbols, confidence, and evidence sentence IDs. The difference is that MarketGyan is localized to the Nepal equity market and avoids claiming trading prediction from generated explanations.

2.3 Parameter-Efficient Fine-Tuning

LoRA adapts large language models by training low-rank update matrices rather than all model weights [8]. QLoRA further reduces memory cost by combining quantization with parameter-efficient fine-tuning [9]. MarketGyan used these methods in stages: Qwen3-8B QLoRA runs were useful for failure analysis, while the final Qwen3.5-9B candidate used BF16 LoRA on an A100 80GB GPU for better structured-output stability.

2.4 Retrieval-Augmented Generation and Citations

RAG combines model generation with retrieved evidence so that answers can reflect current information instead of relying only on model weights [10]. Citation-oriented evaluation work further shows that generated answers should be judged by whether cited sources support claims, not only by fluency [11, 12]. MarketGyan follows this design by materializing citations from retrieved rows and rejecting unsupported public answers.

Chapter 3

Methodology

3.1 System Overview

MarketGyan combines market-data collection, financial-news ingestion, regulatory-document processing, annotation, model training, retrieval, model serving, backend APIs, and frontend demonstration. The methodology keeps these layers separate so that model quality, retrieval quality, and application behavior can be evaluated independently. To keep the architecture readable, the final design is shown as two workflows: the offline training/evaluation workflow in Figure 3.1 and the daily inference workflow in Figure 3.2.

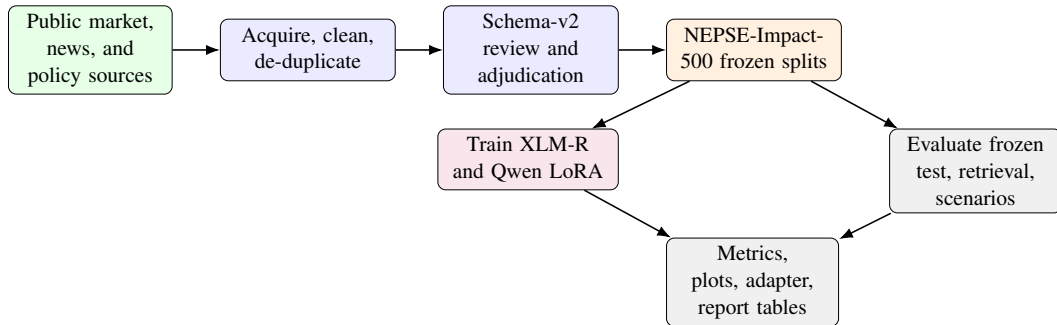


Figure 3.1: Training and evaluation workflow.

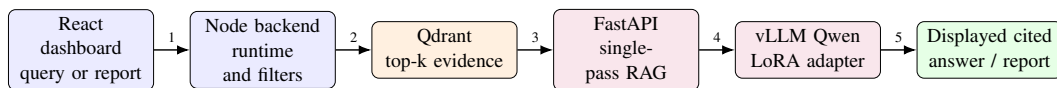


Figure 3.2: Daily inference workflow.

3.2 Data Acquisition Method

The data acquisition strategy follows the proposal’s primary-and-fallback design. For market data, official NEPSE pages are preferred when a field is available in a stable form. ShareSansar and MeroLagani are used as fallback or corroborating sources for current values, listed securities, and historical index backfill. The collector records source provenance, selected source, missing fields, warnings, and conflicts rather than silently merging inconsistent values.

Financial news is collected from ShareSansar, OnlineKhabar Business, and Kathmandu Post Money. ShareSansar is collected through category-specific pagination across NEPSE news, public issues, allotments, dividends, financial analysis, listed shares, and stock-market coverage. OnlineKhabar collection uses paginated Business RSS pages and retrieves full article text when available. Kathmandu Post Money is included as a supplementary source because its historical pagination is limited. Articles shorter than 120 cleaned characters are rejected to avoid media stubs.

Regulatory documents are discovered from NRB and SEBON pages. The final system stores machine-readable text and extraction status, but OCR and legacy-font conversion are excluded from the final deliverable. This keeps the benchmark and RAG index grounded in text that can be inspected and cited.

3.3 Data Pipeline and Storage Contract

The latest final-report snapshot contains 224 historical market snapshots, 2,025 complete financial-news documents, 2,046 eligible documents, 350 active NEPSE symbols, and 2,038 Qdrant evidence chunks in the local MarketGyan collection. Historical market snapshots are marked partial when official breadth, leaders, or sector history fields are unavailable.

Operational storage is split across several contracts. MarketSnapshot stores daily index and turnover metrics, sectors, leaders, provenance, and warnings. MarketDocument stores article or regulatory-document metadata, cleaned text, content hash, source, language, companies, sectors, and extraction method. MarketSecurity stores the active symbol registry and aliases. MarketLabel stores immutable model inputs, generated candidates, numbered source sentences,

token usage, errors, and adjudication state. `MarketAnnotation` stores independent reviewer annotations, and `MarketReaction` stores evaluation-only publication alignment and return windows.

3.4 Benchmark Construction

The first schema version produced a useful pilot but exposed a quality problem: some general-business stories were being forced into bullish, bearish, or neutral impact labels without a defensible NEPSE mechanism. That corpus was frozen as a pilot audit. Schema v2 then separated market relevance from potential impact, added a Nepal-market ontology, required numbered sentence evidence, and supported independent annotations and adjudication.

The final benchmark, NEPSE-Impact-500, contains 500 adjudicated rows:

- 300 direct market-relevant records;
- 100 indirect sector or macro records;
- 100 hard-negative not-relevant records;
- 299 English, 200 Nepali, and 1 mixed-language record;
- 273 symbol-level records; and
- at least 20 records in each required core event category.

The approved JSONL was split into frozen train, validation, and test sets of 350, 75, and 75 rows using balanced duplicate grouping.

3.5 Schema-v2 Curation and Structured Output Contract

Each schema-v2 record preserves the source document, numbered sentences, immutable generated candidate, independent annotations, and final adjudicated gold label. The training export uses the gold label while retaining enough metadata to audit evidence and duplicate groups. The relevant compact JSONL shape is:

```
{
```

```

    "id": "...",
    "title": "...",
    "sentences": [{"id": "S1", "text": "..."}],
    "source": {"name": "...", "url": "..."},
    "publishedAt": "...",
    "contentHash": "...",
    "duplicateGroupId": "...",
    "gold": {
      "relevance": "direct",
      "eventType": "earnings",
      "impactScope": "company",
      "impactDirection": "bullish",
      "impactHorizon": "short_term",
      "impactMechanism": "earnings_cash_flow",
      "sectors": ["Banking"],
      "symbols": ["NABIL"],
      "confidenceBand": "medium",
      "evidenceSentenceIds": ["S1"]
    }
  }
}

```

The compact model output intentionally excludes long rationales and report prose. The generator is asked to emit only the fields needed for classification and evidence selection:

```

{
  "relevance": "direct | indirect | not_relevant",
  "eventType": "market_trading | earnings | regulation | ...",
  "impactScope": "market | sector | company | not_applicable",
  "impactDirection": "bullish | bearish | neutral | ...",
  "impactHorizon": "immediate | short_term | ...",
  "impactMechanism": "market_flow | earnings_cash_flow | ...",
  "sectors": [...],
  "symbols": [...],
  "confidenceBand": "low | medium | high",
  "evidenceSentenceIds": ["S1", "S3"]
}

```

Validation rejects missing fields, unsupported enum values, unregistered symbols,

sectors outside the taxonomy, relevant rows without defensible impact fields, not-relevant rows with inappropriate impact fields, and evidence IDs that are not present in the numbered source sentences. Duplicate groups remain in the same split so that near-duplicate article versions do not leak between train, validation, and test.

3.6 Training Flow

Training examples are generated as two-message chat records. The user message contains the document title and numbered source sentences; the assistant message contains the compact adjudicated JSON output. For XLM-R, title and excerpt are concatenated and used for supervised classification. For Qwen, the chat template is configured so the adapter learns the JSON response rather than prompt text. The frozen test split is never used for oversampling or prompt repair.

3.7 Modeling

The project evaluated a progression of models so that the final Qwen result could be interpreted against simpler baselines. Table 3.1 summarizes the modeling roles.

Table 3.1: Modeling roles used in MarketGyan.

Model path	Role	Reason for inclusion
XLM-R relevance classifier	Multilingual supervised baseline	Tests whether a compact encoder can learn direct, indirect, and not-relevant classes from English and Nepali text.
XLM-R direction classifier	Supervised sentiment/impact baseline	Measures bullish, bearish, neutral, and uncertain direction only on relevant records.
English-only FinBERT	Financial-domain sanity baseline	Tests financial sentiment transfer on the small English relevant subset, while exposing the limitation for Nepali text.
Base Qwen zero-shot and three-shot	Prompting ablation	Measures what the base instruction model can do before domain fine-tuning.
Qwen3-8B QLoRA	Diagnostic LLM adaptation	Exposed invalid JSON and grounding failures under 4-bit QLoRA.
Qwen3.5-9B BF16 LoRA targeted-v2	Final trained candidate	Uses best-validation checkpoint selection and targeted oversampling for structured extraction.

The final adapter was trained on an A100 80GB profile with LoRA rank 32, max sequence length 2,048, batch size 4, gradient accumulation 2, learning rate 2×10^{-5} , and best-validation checkpoint selection. The targeted-v2 oversampling profile expanded the 350 training rows to 922 weighted examples using only the frozen train split, with row weight capped at 8.

3.8 Runtime Design

The initial multi-agent prompt design was replaced for live use because it exceeded vLLM context windows even after increasing context length to 16k tokens. The final application path is a single-pass RAG agent. It retrieves compact evidence rows from Qdrant through the Node backend, sends bounded evidence and market context to the served Qwen3.5-9B adapter, asks the model to choose evidence indexes and draft a response, and then materializes citations from retrieved rows. This design keeps URLs, excerpts, content hashes, sentence IDs, and expanded sentence text under deterministic system control.

Table 3.2: Runtime component responsibilities.

Component	Responsibility
React dashboard	User-facing overview, Ask MarketGyan, Evidence Search, Reports, System, and validation views.
Node backend	Public API, retrieval endpoint, runtime status, report persistence, local report-generation guard, scheduler, and newsletter integration.
Qdrant	Sentence-level finance evidence chunks with source URL, title, source, content hash, sentence IDs, language, symbols, sectors, and tags.
FastAPI agent	Single-pass RAG orchestration, generator call, citation materialization, disclaimer and safety checks.
Lightning vLLM	OpenAI-compatible serving for Qwen/Qwen3.5-9B plus the marketgyan-qwen35-9b-targeted-v2 LoRA adapter.
Evaluation harness	Frozen model scoring, constrained inference scoring, retrieval export/label scoring, and scenario checks.

Chapter 4

System Requirement Specification

4.1 Functional Requirements

The final prototype implements the following proposal-aligned requirements:

- ingest and update finance-specific market data and articles;
- store market documents, snapshots, labels, reports, and security aliases;
- index sentence-level evidence in a vector database;
- fine-tune and evaluate supervised and instruction-model baselines;
- serve the trained Qwen adapter through an OpenAI-compatible endpoint;
- answer finance questions with visible citations and disclaimer text;
- generate local daily market reports with source provenance; and
- maintain scenario, retrieval, and frozen-test evaluation artifacts.

4.2 Non-Functional Requirements

The system must be reproducible, citation-aware, and fail-closed. Model results are measured on frozen train/validation/test splits. Public answers must include disclaimers and must not provide buy or sell advice. Citation objects must include source title, URL, excerpt, chunk ID, content hash, sentence IDs, and expanded sentence text when available. Report generation remains loopback/protected during the demo because it writes archived report records.

4.3 Resource Requirements

Table 4.1: Final resource requirements and actual runtime choices.

Category	Requirement / actual choice
Local development	macOS laptop with Node.js, Python, MongoDB, Docker, and enough storage for article data, model artifacts, and generated report outputs.
Backend and frontend	Existing Khabar AI stack: React, Node.js, Express, MongoDB, Qdrant, Node-Cron, Nodemailer, and Python/FastAPI.
Training hardware	A100 80GB GPU was used for the final Qwen3.5-9B BF16 LoRA run; XLM-R training uses standard transformer fine-tuning resources.
Inference hardware	Lightning AI Studio with vLLM was used for the served Qwen adapter; A100 40GB is sufficient for proposal testing when context length and batch size are controlled, while A100 80GB is safer.
Data resources	NEPSE market summaries, ShareSansar and other finance-news sources, selected NRB/SEBON machine-readable documents, and the adjudicated NEPSE-Impact-500 JSONL benchmark.

4.4 XLM-R Training Configuration

Table 4.2: Implemented XLM-R baseline training configuration.

Item	Configuration
Base model	xlm-roberta-base
Input	Concatenated title and excerpt
Maximum input length	512 tokens
Train / evaluation batch size	8 / 16
Learning rate	2×10^{-5}
Epochs	8 maximum
Weight decay	0.01
Tasks	Relevance on all records; direction on relevant records only
Imbalance handling	Inverse-frequency class-weighted cross-entropy
Selection metric	Validation macro-F1
Early stopping	Patience of 2 epochs
Artifacts	Model, tokenizer, predictions, metrics JSON, and result plots

4.5 Qwen3.5-9B Training Configuration

Table 4.3: Final Qwen3.5-9B targeted-v2 BF16 LoRA training configuration.

Item	Configuration
Base model	Qwen/Qwen3.5-9B
Precision / quantization	BF16 LoRA, no 4-bit loading for the official candidate
Maximum sequence length	2,048 tokens
Maximum generation tokens	256
LoRA rank / alpha / dropout	32 / 64 / 0.05
Train / evaluation batch size	4 / 4 per device
Gradient accumulation	2 steps
Epochs	3
Learning rate	2×10^{-5}
Optimizer	paged_adamw_8bit
Evaluation / save interval	Every 10 steps
Best checkpoint	Loaded by validation loss with greater_is_better=false
Oversampling	targeted_v2; 350 train rows expanded to 922 weighted examples; cap 8
Output	marketgyan-qwen35-9b-a100-80gb- bf16-lora-targeted-v2

Chapter 5

Implementation

5.1 Backend Data Services

MarketGyan is implemented as an extension of the existing Khabar AI backend rather than as a separate standalone server. The Node backend manages market-data ingestion, article records, label/review tooling, runtime status, protected report generation, report persistence, scheduler execution, and newsletter integration. MongoDB stores article, market snapshot, market document, label, annotation, security, and report records. Upserts use stable source URLs and content hashes so that rerunning collectors does not duplicate unchanged documents.

5.2 Annotation and Validation Tools

The data-curation implementation supports three stages: generated candidate creation, independent review, and adjudication. The generated candidate is immutable and used for acceptance/edit-rate analysis. Reviewer edits are stored separately, and only adjudicated gold records enter the v2 export. The validator checks schema version, ontology enums, relevance consistency, sector taxonomy, registered symbols, source metadata, duplicate group IDs, date fields, and sentence-ID grounding. Pending, excluded, v1, and non-adjudicated rows are blocked from training export.

5.3 Vector Indexing and Retrieval

Qdrant stores finance-specific evidence chunks. Each point contains chunk text and metadata such as MongoDB document ID, chunk ID, title, canonical URL, source,

publication date, language, document type, sectors, symbols, tags, content hash, sentence IDs, and expanded sentence text. The indexer is idempotent: unchanged content keeps stable point identifiers, while changed content replaces stale points. Search supports query text plus filters for date, sector, source, language, and document type. The final local collection used for the demo contains 2,038 points.

5.4 Model Serving

Model training and evaluation are implemented in the MarketGyan ML workspace. The XLM-R baseline notebook trains relevance and direction classifiers and exports metrics, predictions, and plots. The Qwen notebook/script path loads the selected profile environment, builds chat-formatted JSON examples, applies optional oversampling, trains the LoRA adapter, saves the best validation checkpoint, exports metrics, writes a training profile, and generates deep error slices. The final adapter is served through Lightning AI using vLLM's OpenAI-compatible endpoint. The active runtime candidate is Qwen/Qwen3.5-9B with the LoRA adapter `marketgyan-qwen35-9b-targeted-v2`. Direct unconstrained prompting is diagnostic only because it can emit reasoning text before JSON. JSON Schema constrained decoding is the correct model-only structured inference path, while the single-pass RAG agent is the correct application path.

5.5 Single-Pass Agent and Reports

The FastAPI agent receives a query or report request, calls the protected Node retrieval endpoint, constructs compact evidence rows, and sends bounded evidence plus market context to the served Qwen adapter. The model returns answer text and selected evidence indexes. The agent then materializes citations from retrieved rows instead of trusting the model to invent URLs or sentence IDs. For reports, the Node backend persists the generated report, citations, sector analysis, and report status. Report generation is local-only in the demo environment because it writes archived records.

5.6 Frontend Demonstration

The React MarketGyan dashboard contains Overview, Ask MarketGyan, Evidence Search, Reports, System, and validation views. The Overview tab shows market snapshot and runtime readiness. Ask MarketGyan sends a grounded query to the backend/agent path and renders disclaimer text, answer text, citations, sentence IDs, and expanded sentence evidence. Evidence Search allows direct retrieval inspection. Reports can force local generation for the latest available snapshot date and display the archived report with sector analysis and evidence citations. The System tab exposes query, review, local report generation, agent-token, Qdrant, snapshot, and latest-report status checks for demo readiness.

5.7 Safety and Fail-Closed Behavior

MarketGyan is informational only. The agent and report paths require disclaimers and reject direct buy or sell advice. If retrieval is weak, if source fields are missing, if the agent token is not configured, or if citation validation fails, the runtime should fail closed rather than publish uncited analysis. This behavior is important because the model content metrics and retrieval Precision@5 are not strong enough to justify unsupported generated claims.

Chapter 6

Results and Evaluation

6.1 Baseline Model Results

The first supervised baseline used XLM-R because it supports both English and Nepali text. It was trained separately for relevance and direction. The relevance classifier uses all 75 frozen test rows, while the direction classifier is evaluated on the 60 relevant test rows. An English-only FinBERT run was also evaluated as a small financial-domain variant on the English relevant subset. Table 6.1 summarizes these baseline outputs.

Table 6.1: XLM-R and FinBERT baseline results.

Model / task	Accuracy	Macro-F1	Errors	Support	Notes
XLM-R relevance	0.787	0.723	16	75	Direct / indirect / not-relevant classification.
XLM-R direction	0.683	0.549	19	60	Bullish / bearish / neutral / uncertain on relevant records.
English-only FinBERT direction	0.889	0.616	3	14	Small English relevant subset; useful only as a sanity baseline.

XLM-R provides a reliable multilingual classifier baseline, but it does not generate structured evidence-grounded outputs. FinBERT shows that financial-domain adaptation helps on English sentiment examples, but its small support and English-only scope make it insufficient for the bilingual MarketGyan task.

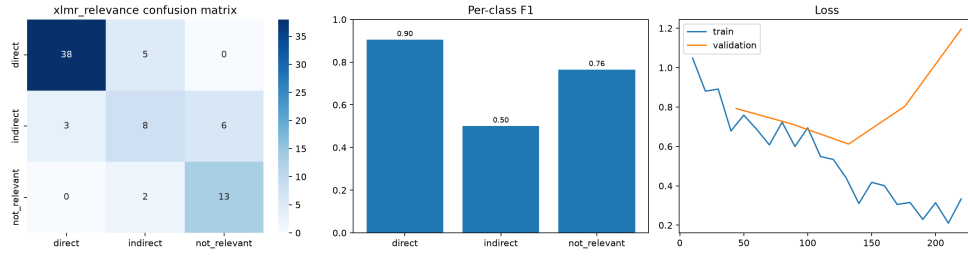


Figure 6.1: XLM-R relevance baseline result plot exported from the v3 baseline artifact.

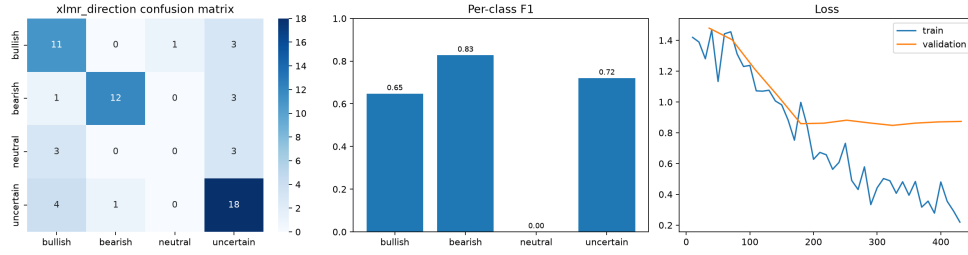


Figure 6.2: XLM-R direction baseline result plot exported from the v3 baseline artifact.

6.2 Qwen Model-Only Benchmark

Table 6.2 summarizes the frozen test-set result for the final Qwen3.5-9B targeted-v2 adapter compared with the previous A100 baseline.

Table 6.2: Strict frozen-test comparison for Qwen3.5-9B A100 runs.

Metric	Baseline	Targeted-v2	Change
Structured-output validity	0.960	1.000	+0.040
Evidence grounding	0.987	1.000	+0.013
Invalid outputs	3	0	-3
Relevance macro-F1	0.735	0.796	+0.061
Nepali relevance accuracy	0.667	0.788	+0.121
Event macro-F1	0.597	0.633	+0.036
Direction macro-F1	0.582	0.575	-0.007
Neutral direction recall	0.000	0.000	0.000
Sector micro-F1	0.759	0.677	-0.082
Symbol micro-F1	0.815	0.793	-0.022
Evidence sentence micro-F1	0.765	0.736	-0.028

The final adapter passes the structural gate and improves relevance, especially for Nepali records. It does not pass the full content gate because event macro-F1 remains below 0.65, direction macro-F1 remains below 0.65, neutral recall is zero,

and set-valued sector, symbol, and evidence metrics remain below target.

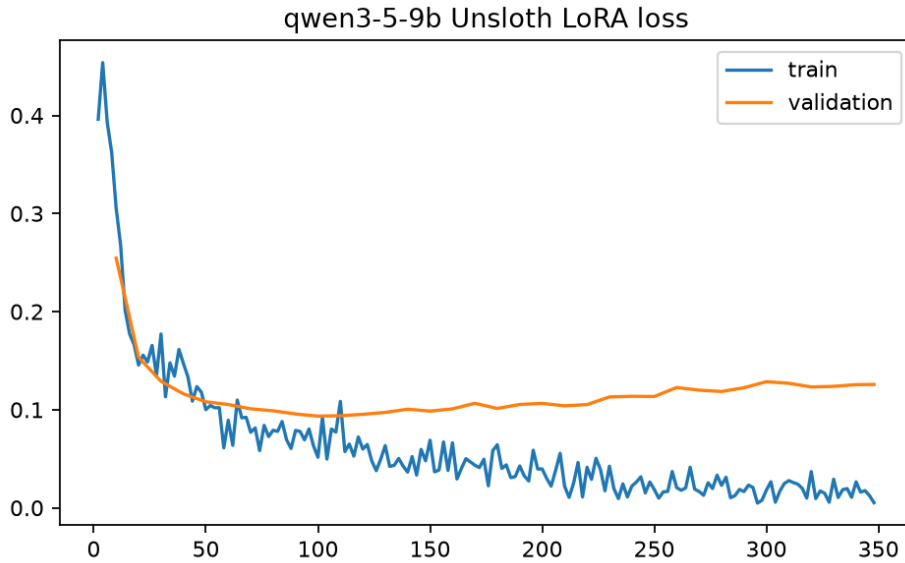


Figure 6.3: Qwen3.5-9B targeted-v2 training and evaluation loss curve from the latest A100 output artifact.

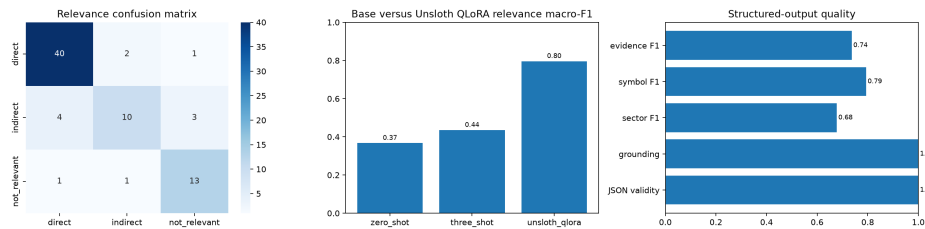


Figure 6.4: Qwen3.5-9B targeted-v2 frozen-test summary plot exported with the final adapter artifact.

6.3 Qwen Prompting and Fine-Tuning Ablation

The Qwen artifact contains zero-shot, three-shot, and fine-tuned adapter predictions on the same frozen 75-row test split. This makes it possible to isolate the effect of prompting versus fine-tuning. Table 6.3 shows that base prompting can sometimes classify event type, but it does not reliably produce valid grounded JSON. Fine-tuning is the main reason strict validity and grounding reach 1.000 and set-field extraction becomes usable.

The ablation shows three important patterns. First, zero-shot base Qwen often emits numeric evidence IDs or malformed outputs, producing zero strict validity. Second,

Table 6.3: Qwen zero-shot, few-shot, and fine-tuned ablation on the frozen test split.

Qwen condition	Validity	Grounding	Invalid	Rel. Acc.	Rel. Macro-F1	Event Macro-F1	Dir. Macro-F1	Sector F1	Symbol F1
Base zero-shot	0.000	0.000	75	0.627	0.367	0.566	0.506	0.013	0.444
Base three-shot	0.987	0.987	1	0.640	0.435	0.532	0.502	0.086	0.380
Qwen3.5-9B LoRA targeted-v2	1.000	1.000	0	0.840	0.796	0.633	0.575	0.677	0.793

Table 6.4: Qwen evidence-selection ablation.

Qwen condition	Evidence F1	Direction acc.	Event acc.
Base zero-shot	0.000	0.567	0.767
Base three-shot	0.599	0.550	0.667
Qwen3.5-9B LoRA targeted-v2	0.736	0.700	0.750

three-shot prompting greatly improves structural behavior but does not improve relevance macro-F1 enough. Third, the targeted-v2 fine-tune improves relevance, evidence selection, sector selection, and symbol selection, but still fails to learn the rare neutral direction class.

6.4 Constrained Runtime Inference

The frozen adapter was also tested through vLLM JSON Schema constrained decoding on the unchanged 75-row test split. Structured validity and evidence grounding remained above the 0.95 gate at 0.987 each. Relevance macro-F1 improved to 0.819, but event, sector, symbol, and evidence metrics declined. This confirms that constrained decoding is useful for schema safety but is not a substitute for better content modeling or retrieval.

Table 6.5: Model-only adapter output versus vLLM JSON Schema constrained inference.

Metric	Adapter output	Constrained vLLM	Change
Structured-output validity	1.000	0.987	-0.013
Evidence grounding	1.000	0.987	-0.013
Invalid outputs	0	1	+1
Relevance macro-F1	0.796	0.819	+0.023
Event macro-F1	0.633	0.604	-0.029
Direction macro-F1	0.575	0.585	+0.010
Sector micro-F1	0.677	0.643	-0.034
Symbol micro-F1	0.793	0.709	-0.084
Evidence sentence micro-F1	0.736	0.704	-0.032

6.5 Retrieval Evaluation

The proposal retrieval suite contains 30 canonical query specifications. The final local Qdrant collection contained 2,038 points, and the retrieval harness exported top-5 candidates for each query. Thirteen queries returned candidates and 17 returned none. Manual labels were added for the 65 returned candidates. The current judged-query Precision@5 is 0.262 over returned query groups. When all 30 canonical queries are counted and empty result sets receive 0/5, the stricter all-query Precision@5 is 0.113. Both values fail the proposal minimum of 0.60. The main observed blocker is missing or mismatched sector/source metadata filters in the indexed payloads.

Table 6.6: Proposal retrieval evaluation after manual labels.

Retrieval item	Result
Canonical queries	30
Queries with candidates	13
Queries with no candidates	17
Total returned candidates	65
Relevant returned candidates	17 / 65
Judged-query Precision@5	0.262
All-query zero-padded Precision@5	0.113
Proposal minimum	0.600

6.6 Scenario and Report Evaluation

The final single-pass live scenario run completed all 20 proposal scenarios. Schema adherence, citation correctness, sentence-citation correctness, grounding, disclaimer presence, and advice-safety checks were all 1.000. Average live latency was 6.665 seconds and p95 latency was 9.735 seconds in the post-smoke run.

Frontend smoke testing also generated a local report for 2026-06-11 with status published, three evidence citations, and three sector-analysis rows. The published report was archived under the internal report identifier [market-gyan://reports/2026-06-11](https://market-gyan.com/reports/2026-06-11).

Table 6.7: Final proposal-aligned system readout.

Area	Result	Decision
Model structure	Validity 1.000, grounding 1.000	Pass
Model content	Relevance 0.796, event 0.633, direction 0.575	Partial; content gate fails
Constrained inference	Validity 0.987, grounding 0.987	Pass for schema safety
Retrieval	Judged P@5 0.262; all-query P@5 0.113	Fail
Live scenarios	20/20 pass for citation and safety checks	Pass for local demo
Frontend/report smoke	Ask, search, and report generation passed	Demo-ready

Chapter 7

Discussion

7.1 Data and Benchmark Findings

The project showed that corpus design matters more than raw model size for this task. The schema-v1 pilot was useful for building tooling, but it mixed true NEPSE-impact stories with broad business coverage and forced many weak examples into bullish, bearish, or neutral labels. Schema v2 corrected this by separating relevance from direction, adding hard-negative and indirect categories, using a controlled event ontology, and requiring evidence sentence IDs. The final NEPSE-Impact-500 benchmark is therefore a stronger research artifact than the early generated labels.

The balanced duplicate-grouped split also mattered. A purely chronological split created a large language/source shift because the article collection timeline differed by outlet. The final balanced split is better for comparing model conditions, while chronological evaluation remains useful as a later robustness test.

7.2 Modeling Findings

XLM-R is the most reliable simple baseline. It performs reasonably on relevance but lacks structured output, evidence selection, and report behavior. English-only FinBERT confirms the value of financial-domain adaptation, but its coverage is too narrow for Nepali and mixed-language MarketGyan inputs. The Qwen3-8B QLoRA runs were important failure-analysis artifacts: they showed that a model can partially learn extraction while still failing strict JSON validity and evidence grounding.

The final Qwen3.5-9B targeted-v2 run solved the structural-output problem and im-

proved relevance, especially for Nepali examples. However, it did not solve all content errors. Neutral direction remains the clearest modeling failure, with zero recall. Sector, symbol, and evidence sentence metrics show a recall problem: the model usually selects valid items but often selects too few. This suggests that the next model improvement should come from better labeled coverage, clearer evidence-selection rules, or constrained post-processing rather than another blind training run.

7.3 Retrieval and Runtime Findings

The runtime evaluation separated model behavior from system behavior. JSON Schema constrained decoding is useful when the goal is valid structured output, but it does not automatically improve event, sector, symbol, or evidence F1. The single-pass RAG agent was a practical engineering improvement over the earlier CrewAI-style multi-agent prompt because it avoided repeated context-window failures and kept citations under deterministic control.

The largest remaining system weakness is retrieval. The Qdrant collection contains sentence-level evidence and the frontend can demonstrate evidence search, but the canonical retrieval benchmark fails because many proposal queries return no candidates under sector or source filters. This is a metadata and indexing problem, not a training problem. Fixing sector aliases, source names, document-type values, and ticker metadata should come before any additional model training.

7.4 Final Project Status

The final status is **demo-only / research prototype**. The system can be shown as a grounded demonstration of Nepal-market financial NLP, structured Qwen adaptation, RAG citation materialization, evidence search, and local report generation. It should not be presented as a deployable market-analysis or advisory product until retrieval Precision@5 improves, neutral/direction errors are reduced, and set-valued sector, symbol, and evidence recall are stronger.

Chapter 8

Limitations and Future Work

8.1 Limitations

- Historical market snapshots are partial and do not contain complete official fields for every date.
- NEPSE-Impact-500 is useful but small; rare event and neutral classes are still under-covered.
- Retrieval filters need stronger metadata normalization for sector, source, document type, and ticker aliases.
- Independent second-review agreement has not yet been reported.
- Market-reaction analysis remains exploratory and should not be interpreted as causal forecasting.
- Lightning-hosted vLLM is suitable for demonstration but is not a production deployment architecture by itself.

8.2 Future Work

1. fix retrieval metadata normalization and rerun the 30-query Precision@5 benchmark;
2. complete second annotation and report agreement metrics;
3. add or adjudicate more neutral, sector-industry, regulation, credit-financing, and ticker hard cases;

4. evaluate market-reaction associations with missing-data and overlap warnings;
5. keep JSON Schema constrained inference for model-only structured use; and
6. keep the single-pass RAG path as the production candidate unless a shorter multi-step agent can pass context and citation gates.

Chapter 9

Conclusion

MarketGyan successfully extends Khabar AI into a Nepal-specific financial NLP research prototype. It contributes NEPSE-Impact-500, a bilingual evidence-grounded benchmark; a Qwen3.5-9B targeted-v2 LoRA adapter; a finance-specific Qdrant evidence index; a single-pass RAG agent; and a React dashboard capable of demonstrating grounded query and report generation. The final model passes strict structure and grounding gates and the live system passes proposal scenario citation checks. However, retrieval Precision@5 and several content metrics remain below the proposal gates. The project should therefore be submitted as a strong demo-ready research prototype with clear remaining work, not as a deployable financial advisory product.

Chapter A

Demo Screenshots

The screenshots in this appendix are extracted from the final local Qwen demo recording included with this submission in the demo-recording directory as marketgyan-qwen-demo.mp4. They document the frontend path used for the final smoke test: overview, Ask MarketGyan, generated answer with evidence, and report generation.

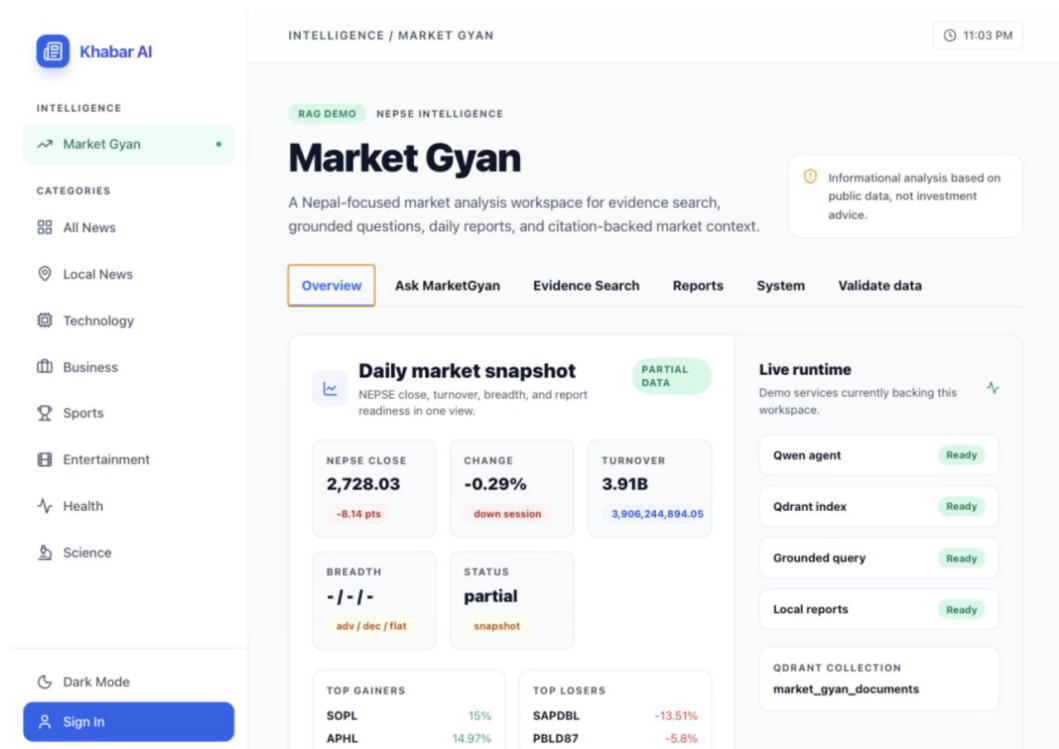


Figure A.1: MarketGyan overview tab showing market snapshot, live runtime checks, and Qdrant collection status.

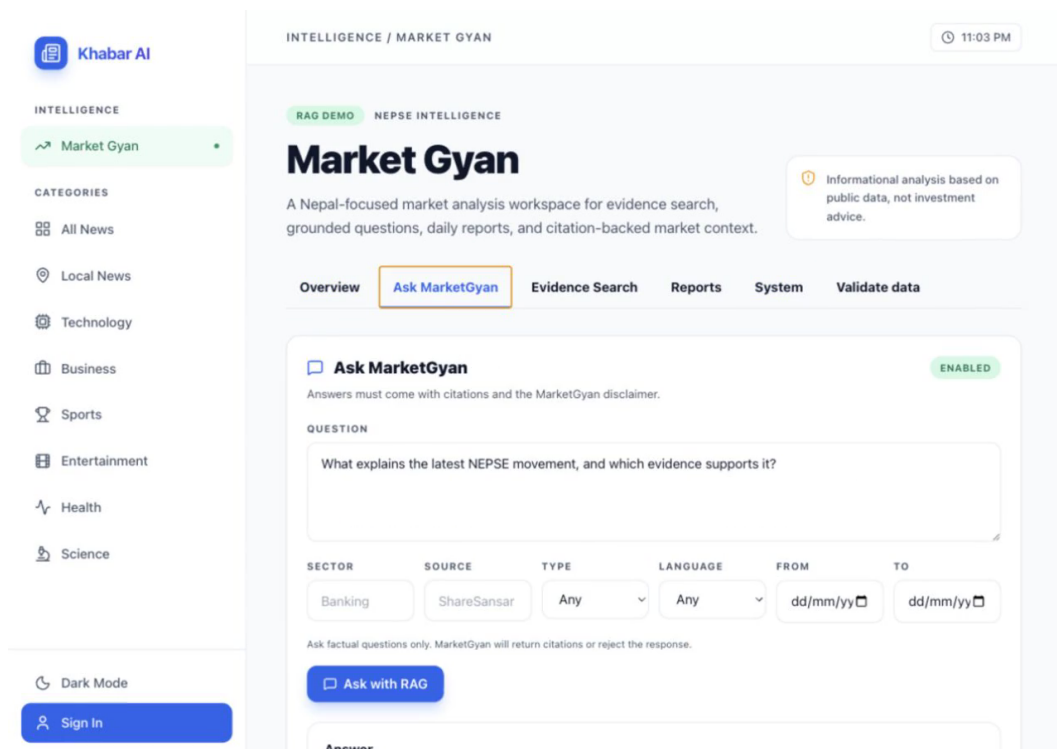


Figure A.2: Ask MarketGyan tab with a proposal-style market query form.

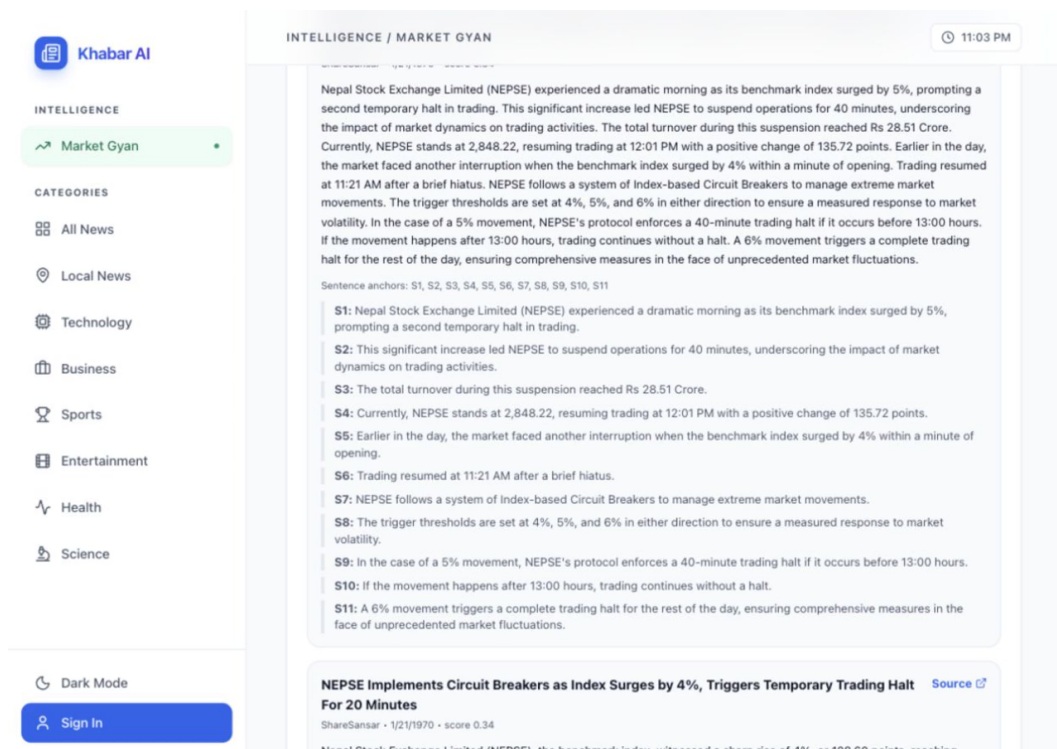


Figure A.3: Generated Qwen answer with disclaimer, sentence-grounded citations, and source evidence.

Khabar AI

INTELLIGENCE

Market Gyan

CATEGORIES

All News

Local News

Technology

Business

Sports

Entertainment

Health

Science

Dark Mode

Sign In

INTELLIGENCE / MARKET GYAN

11:03 PM

Nepal Stock Exchange Market Snapshot: Mixed Performance Amid Regulatory Updates

6/11/2026 • published • marketgyan-qwen35-9b-targeted-v2

The Nepal Stock Exchange (NEPSE) closed at 2728.03 points, down 0.29% on the day. Market activity showed mixed performance with significant gains in hydropower and pharmaceutical sectors, while debentures and finance companies faced declines. Trading volumes were led by City Hotel Limited and Peoples Hydropower Company Limited. Recent regulatory updates regarding NEPSE trading hours and corporate divestment announcements continue to shape market dynamics.

Sector analysis

Hydropower

Bullish

Hydropower companies led the market gains, with Upper Lohore Khola Hydropower Company Limited (ULHC) rising 6.07% and Peoples Hydropower Company Limited (PHCL) up 5.33%. High trading volumes in PHCL (494,194) and CITY (556,409) indicate strong investor interest in this sector.

Finance

Bearish

Finance companies and debentures experienced significant declines. Saptakoshi Development Bank Limited (SAPDBL) dropped 13.51%, while Best Finance Company Limited (BFC) fell 3.59%. Debentures from Prabhu Bank and Nepal SBI Bank also saw notable losses.

Regulatory Environment

Neutral

Recent regulatory changes, including NEPSE trading hour adjustments and corporate divestment announcements, continue to influence market structure. The resumption of regular trading hours under new government timings provides clarity for market participants.

Report citations

RU RU Jalbidhyut Pariyojana Limited Announces Partial Divestment from Karnali Water Power Pvt. Ltd.

Source

ShareSansar • 6/16/2025 • score 0.50

This decision aligns with the regulatory requirement to inform the Nepal Stock Exchange and maintain transparency for

Figure A.4: Published MarketGyan report with headline, market summary, sector analysis, and report citations.

32

References

- [1] D. Araci, “FinBERT: Financial Sentiment Analysis with Pre-trained Language Models,” arXiv preprint arXiv:1908.10063, 2019.
- [2] H. Yang et al., “FinGPT: Open-Source Financial Large Language Models,” arXiv preprint arXiv:2306.06031, 2023.
- [3] J. Lee, N. Stevens, S. C. Han, and M. Song, “A Survey of Large Language Models in Finance,” *Neural Computing and Applications*, 2024.
- [4] G. Bhatia et al., “FinTral: A Family of GPT-4 Level Multimodal Financial Large Language Models,” arXiv preprint arXiv:2402.10986, 2024.
- [5] S. Timilsina, M. Gautam, and B. Bhattarai, “NepBERTa: Nepali Language Model Trained in a Large Corpus,” in *Proceedings of ACL-IJCNLP*, 2022, pp. 273–284.
- [6] B. Subedi, S. Regmi, B. K. Bal, and P. Acharya, “Exploring the Potential of Large Language Models for Low-resource Languages: A Study on Named-Entity Recognition and Part-of-Speech Tagging for Nepali Language,” in *LREC-COLING*, 2024, pp. 6974–6979.
- [7] P. Thapa, J. Nyachhyon, M. Sharma, and B. K. Bal, “Development of Pre-Trained Transformer-based Models for the Nepali Language,” in *CHiPSAL*, 2025, pp. 9–19.
- [8] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *International Conference on Learning Representations*, 2022.
- [9] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

- [10] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [11] T. Gao, H. Yen, J. Yu, and D. Chen, “Enabling Large Language Models to Generate Text with Citations,” in *Proceedings of EMNLP*, 2023.
- [12] J. Zhang et al., “LongCite: Enabling LLMs to Generate Fine-Grained Citations in Long-Context QA,” arXiv preprint arXiv:2409.02897, 2024.
- [13] Qwen Team, “Qwen3.5-9B Model Card,” Hugging Face, 2026. [Online]. Available: <https://huggingface.co/Qwen/Qwen3.5-9B>.
- [14] vLLM Project, “OpenAI-Compatible Server and Structured Outputs,” 2026. [Online]. Available: <https://docs.vllm.ai/>.
- [15] Nepal Stock Exchange, “NEPSE Market Data,” 2026. [Online]. Available: <https://nepalstock.com/>.
- [16] ShareSansar, “NEPSE Market and Index History Data,” 2026. [Online]. Available: <https://www.sharesansar.com/>.
- [17] Nepal Rastra Bank, “Notices, Publications, and Policy Documents,” 2026. [Online]. Available: <https://www.nrb.org.np/>.
- [18] Securities Board of Nepal, “Notices and Regulatory Publications,” 2026. [Online]. Available: <https://www.sebon.gov.np/>.